

Laiba Taseen

23-NTU-CS-1258

Embedded IOT Systems

Assignment 1:

Question 1 — Short Questions Answer concisely, in your own words.

1. Why is volatile used for variables shared with ISRs?

Because ISR (Interrupt Service Routine) variables can change unexpectedly, the volatile keyword prevents the compiler from optimizing them out — ensuring the main code always reads the latest value from memory.

2. Compare hardware-timer ISR debouncing vs. delay()-based debouncing.

- **Hardware-timer ISR debouncing:** Uses precise timer interrupts; non-blocking and efficient.
 - **delay()-based debouncing:** Uses software delays; blocks execution and is less accurate.
- Hardware-timer methods are preferred for real-time and multitasking systems.

3. What does IRAM_ATTR do, and why is it needed?

IRAM_ATTR places a function (like an ISR) in **Instruction RAM (IRAM)** instead of flash memory. It's needed because ISRs must run quickly and flash access may be too slow or disabled during interrupts.

4. Define LEDC channels, timers, and duty cycle.

- **Channels:** Control individual PWM outputs.
- **Timers:** Set the PWM frequency and resolution.
- **Duty cycle:** The percentage of time the signal stays HIGH in one PWM period, controlling brightness or speed.

5. Why should you avoid Serial prints or long code paths inside ISRs?

Because ISRs must execute very quickly. Serial prints or long operations can delay other interrupts, cause missed events, or crash the system.

6. What are the advantages of timer-based task scheduling?

- Precise and consistent timing

- Non-blocking execution
- Better multitasking and CPU efficiency
- Improved control over periodic tasks like sampling or blinking LEDs

7. Describe I²C signals SDA and SCL.

- **SDA (Serial Data):** Carries the data between devices.
- **SCL (Serial Clock):** Provides the timing for data transfer, generated by the master device.

8. What is the difference between polling and interrupt-driven input?

- **Polling:** Continuously checks the input in a loop (CPU-intensive).
- **Interrupt-driven:** Reacts automatically when an event occurs (efficient, saves CPU time).

9. What is contact bounce, and why must it be handled?

When a mechanical switch is pressed or released, its contacts may rapidly open and close before settling. This “bounce” creates multiple false signals and must be debounced to ensure only one clean input.

10. How does the LEDC peripheral improve PWM precision?

The **LEDC (LED Controller)** uses hardware timers and high-resolution counters to generate stable, accurate PWM signals — providing fine control over frequency and duty cycle without heavy CPU load.

11. How many hardware timers are available on the ESP32?

The ESP32 has **4 hardware timers per group** and **2 timer groups**, giving a total of **8 hardware timers** that can be used independently.

12. What is a timer prescaler, and why is it used?

A **prescaler** divides the main clock frequency before it reaches the timer, slowing it down. This allows longer timing intervals and more flexible timing configurations.

13. Define duty cycle and frequency in PWM.

- **Duty cycle:** The percentage of one PWM period when the signal is HIGH.
- **Frequency:** How many PWM cycles occur per second (measured in hertz, Hz).

14.How do you compute duty for a given brightness level?

$\text{Duty} = (\text{Brightness level} / \text{Max level}) * \text{Max duty level}$

For example, with 8-bit resolution:

```
duty = (brightness / 255) * 255.
```

15.Contrast non-blocking vs. blocking timing.

- **Blocking timing:** Uses delays (`delay()`), stopping other tasks until complete.
- **Non-blocking timing:** Uses `millis()`, timers, or interrupts, allowing the CPU to perform other tasks simultaneously.

16.What resolution (bits) does LEDC support?

The LEDC peripheral supports up to **20-bit resolution**, allowing very fine PWM control depending on frequency settings.

17.Compare general-purpose hardware timers and LEDC (PWM) timers.

- **Hardware timers:** General timing control for interrupts, delays, or scheduling.
- **LEDC timers:** Dedicated to PWM signal generation with adjustable frequency and resolution.
LEDC is optimized for tasks like LED dimming and motor control.

18.What is the difference between Adafruit_SSD1306 and Adafruit_GFX?

- **adafruit_SSD1306:** A **display driver library** specifically for OLEDs using the SSD1306 controller.
- **Adafruit_GFX:** A **graphics library** that provides drawing functions (text, shapes) used by many display drivers including SSD1306.

19.How can you optimize text rendering performance on an OLED?

- Minimize screen refreshes (update only changed areas).
- Use smaller fonts or pre-rendered bitmaps.
- Reduce I²C communication by buffering changes in memory before updating.
- Avoid repeatedly clearing the screen unnecessarily

20. Give short specifications of your selected ESP32 board (NodeMCU-32S).

- **Microcontroller:** ESP32-D0WDQ6 (dual-core 32-bit Xtensa LX6 CPU)
- **Clock Speed:** Up to 240 MHz
- **Flash Memory:** 4 MB
- **SRAM:** 520 KB
- **Wi-Fi:** 802.11 b/g/n (2.4 GHz)
- **Bluetooth:** v4.2 BR/EDR and BLE
- **GPIO Pins:** 30 (including ADC, DAC, PWM, I²C, SPI, UART)
- **Analog Inputs:** 18-channel 12-bit ADC
- **DAC Outputs:** 2 (8-bit)
- **Operating Voltage:** 3.3 V (5 V via USB)
- **USB Interface:** Micro-USB (via CP2102/CH340 USB-to-Serial chip)
- **Power Supply:** 5 V (USB) or 3.3 V (VIN pin)
- **Other Features:** On-board antenna, reset & boot buttons, built-in voltage regulator

Question 2 — Logical Questions

1. A 10 kHz signal has an ON time of 10 ms. What is the duty cycle? Justify with the formula.

Step 1 — check consistency.

- Period $T = 1/f = 1/10,000 = 0.0001 \text{ s} = 0.1 \text{ ms} = 100 \mu\text{s}$.
- Given ON time = 10 ms = 10,000 μs
- , which is 100× the period. That means ON time > period — impossible for a single-cycle duty (it gives >100%).

Conclusion (as given):

$$\text{Duty cycle} = \frac{t_{\text{ON}}}{T} \times 100\% = \frac{10 \text{ ms}}{0.1 \text{ ms}} \times 100\% = 10,000\%.$$

This indicates the input values are inconsistent — either the frequency or the ON time is wrong.

Likely intended correction: if ON time was 10 μs (microseconds) not 10 ms, then $T = 100 \mu\text{s}$, $t_{\text{ON}} = 10 \mu\text{s} \rightarrow \text{duty} = 10/100 \times 100\% = 10\%$.

2. How many hardware interrupts and timers can be used concurrently? Justify.

General-purpose hardware timers: ESP32 has **two timer groups · two timers per group=4 independent hardware timers for general-purpose use.**

Interrupts: The ESP32 interrupt architecture exposes **two cores with 32 interrupt entries each** (i.e., an interrupt allocation space of 32 per core). There are more interrupt *sources* than interrupt *entries*, so drivers sometimes share entries; `esp_intr_alloc()` handles allocation. In practice you can register many ISR sources, but they map into the available interrupt entries and priorities.

3. How many PWM-driven devices can run at distinct frequencies at the same time on ESP32? Explain constraints.

- **Number of PWM channels (independent outputs / duty controls):** ESP32 SoC provides **up to 16 LEDC channels** (arranged as two groups of 8 channels). Each channel controls duty/GPIO mapping.
- **Number of distinct frequencies:** determined by the number of **LEDC timers/clock modules** (each LEDC *timer/clock* defines frequency + resolution for the channels attached to it). The LEDC implementation offers multiple clock modules: four high-speed and four low-speed clock modules (so up to **8 selectable timer/clock sources** across HS and LS), meaning **up to ~8 distinct frequencies** (practical number depends on SoC variant and driver). Each distinct frequency consumes one LEDC timer/clock; multiple channels can share the same timer (same frequency) but have independent duty/resolution.

4. Compare a 30% duty cycle at 8-bit resolution and 1 kHz to a 30% duty cycle at 10-bit resolution (all else equal).

- **8-bit resolution:** max count = 255. 30% duty $\rightarrow 0.30 \times 255 = 76.5$. Practical duty count ≈ 77 (or 76 if rounding down).
- **10-bit resolution:** max count = 1023. 30% duty $\rightarrow 0.30 \times 1023 = 306.9$. Practical duty count ≈ 307 .

Comparison / effect: both represent the *same percentage* (30%) so the **perceived output (average voltage / brightness)** is the same at 1 kHz. The difference: **10-bit gives finer granularity** (smaller step size) — steps are $1/1023 \approx 0.098\%$ vs $1/255 \approx 0.392\%$ per LSB, so 10-bit lets you adjust brightness more smoothly and reduces quantization error. Frequency (1 kHz) equal in both, so no difference there.

5. How many characters can be displayed on a 128×64 OLED at once with the minimum font size vs. the maximum font size? State assumptions.

Assumption A — small 5×7 font with 1-pixel spacing (typical “6×8” cell):

- Character cell = 6×8 pixels (width $\times$ height).
- Columns = $\text{floor}(128 / 6) = \text{floor}(21.333\dots) = 21$ columns.
- Rows = $\text{floor}(64 / 8) = 8$ rows.
- **Total chars = $21 \times 8 = 168$ characters.**

Assumption B — common medium font (8×16 cell):

- Columns = $\text{floor}(128 / 8) = 16$.
- Rows = $\text{floor}(64 / 16) = 4$.
- **Total chars = $16 \times 4 = 64$ characters.**

Assumption C — very large font (16×32 cell):

- Columns = $\text{floor}(128 / 16) = 8$.
- Rows = $\text{floor}(64 / 32) = 2$.
- **Total chars = $8 \times 2 = 16$ characters.**

Notes / summary: pick the font/cell size you mean:

- **Minimum-font (6×8) → ≈ 168 chars on 128×64.**
- **Medium (8×16) → 64 chars.**
- **Large (16×32) → 16 chars.**